

Thread Methods In Java

1. Thread.sleep()

- **Definition:**
Makes the current thread pause execution for a specified time (in milliseconds).
- **Syntax:**

```
Thread.sleep(1000); // 1000ms = 1 second
```

Key Points:

- Thread goes into **Timed Waiting State**.
- After the time, thread becomes **Runnable** again.
- It throws **InterruptedException**, so must be handled.

2. Thread.yield()

- **Definition:**
Hints the scheduler that the current thread is willing to give up its CPU to let other threads execute.
- **Syntax:**

```
Thread.yield();
```

Key Points:

- It does **not guarantee** that CPU will shift.
- Used to improve performance in some multi-threaded programs.
- Thread remains in **Runnable state** (not blocked).

3. Thread.stop() (⚠️ Deprecated)

- **Definition:**
Terminates the thread execution immediately.
- **Syntax:**

```
thread.stop(); // Not recommended
```

Key Points:

- **Unsafe method** → may cause inconsistent state because it stops thread without cleanup.
- Removed from practice in modern Java.
- Better way: use a **flag variable** to stop a thread gracefully.

4. Thread Priorities

- **Definition:**

Each thread has a priority that helps the thread scheduler decide which thread to run first.

- **Values:**

- MIN_PRIORITY = 1
- NORM_PRIORITY = 5 (default)
- MAX_PRIORITY = 10

- **Syntax:**

```
Thread t1 = new Thread();

t1.setPriority(Thread.MAX_PRIORITY); // 10

t1.setPriority(Thread.MIN_PRIORITY); // 1
```

Key Points:

- Higher priority threads may get preference, but scheduling depends on JVM and OS.
- Not a guarantee.

```
class MyThread extends Thread {

    public void run() {

        for(int i=1; i<=5; i++) {

            System.out.println(getName() + " running: " + i);

            try {

                Thread.sleep(500); // pause 0.5 sec

            } catch (InterruptedException e) {

                e.printStackTrace();

            }

            Thread.yield(); // give chance to others

        }

    }

}
```

```
public class ThreadMethodsDemo {

    public static void main(String[] args) {

        MyThread t1 = new MyThread();

        MyThread t2 = new MyThread();

    }

}
```

```
t1.setPriority(Thread.MAX_PRIORITY); // 10
```

```
t2.setPriority(Thread.MIN_PRIORITY); // 1
```

```
t1.start();
```

```
t2.start();
```

```
// t1.stop(); // ✗ Deprecat
```

```
}
```

```
}
```